

令和 8 年度

卒業論文

リカレントニューラルネットワークを用いた
ニューロンモデルの代理モデルの構築

指導教員 小林 泰良 助教

山口大学

理学部 物理・情報科学科

情報科学コース 4 年

山下 陽平

目次

第 1 章	はじめに	2
第 2 章	基礎知識	4
2.1	ニューロンのモデル化	4
2.2	Hodgkin-Huxley モデル	4
2.3	Multi-Compartment モデル	5
2.4	リカレントニューラルネットワーク	6
第 3 章	RNN による代理モデルの構築手法	10
3.1	概要	10
3.2	シミュレーション条件	10
3.3	学習データの作成	12
3.4	RNN の実装	12
3.5	代理モデルとしての推論方法	13
3.6	評価指標	14
第 4 章	結果と考察	16
4.1	sin 波による RNN 実装の検証	16
4.2	Hodgkin-Huxley モデルのシミュレーション	17
4.3	学習結果	17
4.4	代理モデルの評価	17
4.5	考察	17
第 5 章	まとめ	19
	謝辞	20
	参考文献	21
第 A 章	付録	23
A.1	実装したコード	23

第 1 章

はじめに

脳は生物の思考や行動といった情報処理を司る器官であり、その機能はニューロンのネットワーク上を伝播するスパイクによって実現される [1]。脳の情報処理の仕組みを知るためにはネットワーク上のスパイクを観測する必要があるが、実際の脳は莫大な数 (10^{11} 個程度 [2]) のニューロンから構成されるため、そのすべてを同時に観測することは極めて難しい。一方で、個別のニューロンの振る舞いについては多くのことが知られている。そのため、ニューロンの振る舞いを記述する数理モデルを組み合わせた脳モデルを構築し、計算機上でシミュレーションを行うという手法が広く用いられている。

近年の研究によると、ニューロンは刺激シーケンスの弁別や排他的論理和演算といった複雑な演算を行っており、これにはニューロンそのものが持つ空間形状が大きく関わっていることが分かってきた [3, 4]。ニューロンの空間形状をモデル化に組み入れた数理モデルが Multi-Compartment モデルであり、このモデルはニューロンを電氣的に連結された複数の区画（コンパートメント）の群として表現することで、上記のような特性を忠実に再現する。しかし、Multi-Compartment モデルは各コンパートメントが膜電位と複数の隠れ変数を状態として保持するため、シミュレーションに多くのメモリを必要とする [1]。マウスの脳（約 $10^8 \sim 10^9$ 個のニューロンを持つ）程度の規模であっても、国内最高性能のスーパーコンピュータである富岳でも扱いきれないほどに空間計算量が増大してしまう [5]。脳シミュレーションにおいては、隠れ変数の値そのものを直接知る必要はないにもかかわらず、隠れ変数を格納するためのメモリはニューロン数とコンパートメント数に比例して増加してしまう。したがって、大規模な脳モデルのシミュレーションを実現するためには、ニューロンモデルをシミュレーションする際の計算コストを削減することが重要である。

計算コストを削減する手法として、本研究では代理モデル（surrogate model）の構築を考える。ここでの代理モデルとは、元のニューロンモデルに対し、より少ない隠れ変数、あるいはより高速な計算で同等の膜電位応答を再現できるモデルを指す。棟近先輩による先行研究では、時系列データから支配方程式をスパースに同定する手法である SINDy [9] と主成分分析を組み合わせることで、Hodgkin-Huxley モデル [6] の 4 つの状態変数 (V, m, h, n) を 2 つ (V, g') に圧縮した代理モデルが構築されている [10]。この手法では空間計算量の削減には成功したが、同定された微分方程式の右辺の項数が十分にスパースにならず、時間計算量はかえって増加するという課題が残されている。

そこで本研究では、代理モデルの構築手法としてリカレントニューラルネットワーク（Recurrent

Neural Network, 以下 RNN) を用いる。RNN を用いる理由は次の 3 点である。

- (1) **内部状態を保持できる**。ニューロンモデルにおける現在の膜電位は、現在の入力だけでなく過去の膜電位や状態変数にも依存する。RNN は隠れ状態として過去の情報を保持するため、このような動的システムの状態遷移を自然に表現できる。
- (2) **時系列データとの親和性が高い**。膜電位は時間とともに変化する時系列データであり、RNN は時系列データの時間相関を学習するために設計されたモデルである。
- (3) **GPU による高速化が期待できる**。RNN の推論は主に行列積で構成されるため、GPU による並列化と相性がよい。微分方程式を逐次的に数値積分する従来手法に対し、高速な推論が期待できる。

本研究の最終的な目標は Multi-Compartment モデルに対する代理モデルの構築であるが、Multi-Compartment モデルは空間形状を考慮した複雑なモデルであるため、いきなりこれを対象とするのは難易度が高い。そこで、より基本的なニューロンモデルで RNN による代理モデルが機能するかをまず確かめる。そのようなモデルとして、空間形状を考慮しないが Multi-Compartment モデルと同様にイオン輸送を制御する隠れ変数を持つ Hodgkin-Huxley モデルを選ぶ [6]。

また本研究では、RNN の内部で行われる計算を理解することを重視し、PyTorch などの深層学習フレームワークを用いず、NumPy の基本的な配列演算のみで RNN を実装する。これにより、順伝播・誤差逆伝播・パラメータ更新の各過程をすべて追跡可能な形で記述する。

第 2 章

基礎知識

2.1 ニューロンのモデル化

脳は感覚情報の処理、記憶、思考、運動司令などを司る情報処理器官であり、これはニューロンのネットワークによって実現される。ニューロンは電流の入力を受けると、細胞の内外の電位差である膜電位を変化させる。膜電位の変化には、細胞膜と、細胞膜に埋め込まれたイオンチャネル（特定のイオンのみを選択的に透過させる輸送体）が深く関わっている。

ニューロンの電氣的な振る舞いは、細胞膜を容量 C [$\mu\text{F}/\text{cm}^2$] のキャパシタ、膜を透過するイオンの流れ I_{leak} [$\mu\text{A}/\text{cm}^2$] を内部コンダクタンス g_{leak} [mS/cm^2]、起電力 E_{eq} [mV] の電池を流れる電流と見立てることで、RC 回路とみなすことができる [7]。外部から入力される電流を I_{ext} [$\mu\text{A}/\text{cm}^2$] とすると、膜電位 V の時間変化は次の微分方程式に従う。

$$C \frac{dV}{dt} = -g_{\text{leak}} (V(t) - E_{\text{eq}}) + I_{\text{ext}}(t) \quad (2.1)$$

このモデルは細胞膜の基本的なダイナミクスを記述しているが、これだけでは膜電位が上下するのみでスパイクは発射されない。イオンチャネルの種類まで考慮してモデル化することで、スパイクの発射を再現できる。

[TODO: 図 2.1 として RC 回路図を入れる。TikZ で描くか、先輩の卒論の図を参考に自作する]

2.2 Hodgkin-Huxley モデル

単一ニューロンの数理モデルの中で世界で最初に開発されたものが Hodgkin-Huxley モデル（以下 HH モデル）である [6]。HH モデルは、イオン電流をナトリウムイオン電流 I_{Na} 、カリウムイオン電流 I_{K} 、リーク電流 I_{leak} （その他のイオン電流）に分離したモデルである。式 (2.1) のイオン電流の項を分割することで、膜電位 V [mV] の変化式は次のように書ける。

$$C \frac{dV}{dt} = -g_{\text{leak}} (V(t) - E_{\text{leak}}) - g_{\text{Na}}(V, t) (V(t) - E_{\text{Na}}) - g_{\text{K}}(V, t) (V(t) - E_{\text{K}}) + I_{\text{ext}}(t) \quad (2.2)$$

ここで、ナトリウムコンダクタンス $g_{\text{Na}}(V, t)$ とカリウムコンダクタンス $g_{\text{K}}(V, t)$ は V, t に従って

変化する。

$$g_{\text{Na}}(V, t) = \bar{g}_{\text{Na}} m^3(V, t) h(V, t) \quad (2.3)$$

$$g_{\text{K}}(V, t) = \bar{g}_{\text{K}} n^4(V, t) \quad (2.4)$$

$\bar{g}_{\text{Na}}, \bar{g}_{\text{K}}$ は各チャネルのコンダクタンスがとりうる最大値を表す定数（表 2.1）であり、 m, h, n は膜電位に依存して変化するゲート変数である。ゲート変数はイオンチャネルの開口率を表す $0 \sim 1$ の値をとる変数であり、膜電位と同様に時間に関する一階の微分方程式に従う。

$$\frac{d}{dt}x(V, t) = \alpha_x(V) (1 - x(V, t)) - \beta_x(V) x(V, t) \quad (x = m, h, n) \quad (2.5)$$

$\alpha_x(V), \beta_x(V)$ はそれぞれ次式で定義される。

$$\begin{aligned} \alpha_m(V) &= \frac{2.5 - 0.1V}{\exp(2.5 - 0.1V) - 1}, & \beta_m(V) &= 4 \exp\left(-\frac{V}{18}\right) \\ \alpha_h(V) &= 0.07 \exp\left(-\frac{V}{20}\right), & \beta_h(V) &= \frac{1}{\exp(3 - 0.1V) + 1} \\ \alpha_n(V) &= \frac{0.1 - 0.001V}{\exp(1 - 0.1V) - 1}, & \beta_n(V) &= 0.125 \exp\left(-\frac{V}{80}\right) \end{aligned} \quad (2.6)$$

まとめると、HH モデルは膜電位 V とゲート変数 m, h, n に関する 4 元の連立常微分方程式である。

表 2.1 Hodgkin-Huxley モデルで使うイオンチャネルに関する定数

イオン種	反転電位 E [mV]	最大コンダクタンス $\bar{g}$ [mS/cm ²]
Na ⁺	115.0	120.0
K ⁺	-12.0	36.0
その他のイオン (leak)	10.6	0.3

2.3 Multi-Compartment モデル

Multi-Compartment モデルは、ニューロンの空間形状を電氣的に連結された複数のコンパートメントとして表現したニューロンモデルである。各コンパートメントはそれぞれ膜電位と状態変数を持ち、隣接するコンパートメントとの間には電位差に比例した軸方向電流が流れる。コンパートメント i に流れ込む軸方向電流は次式で表される。

$$I_i^{(\text{axial})} = \sum_{j \in \text{neighbors}} g_{i,j} (V_j - V_i) + I_{\text{inj}} \quad (2.7)$$

[TODO: どの Multi-Compartment モデルを対象とするかが決まっていない。小林先生から紹介された論文を読んで、対象とするモデル（コンパートメント数、各コンパートメントの種類）を確定させ、ここに記述する]

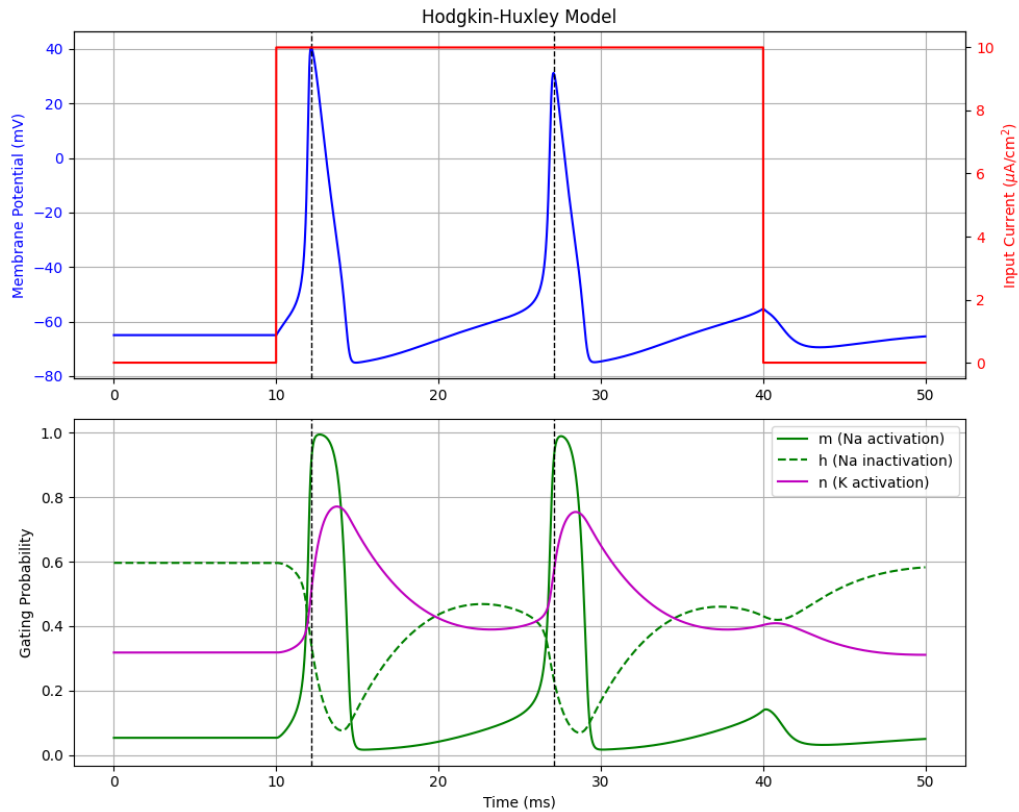


図 2.1 Hodgkin-Huxley モデルのシミュレーション結果。[TODO: 図の説明を書く。与えた電流の振幅・時間、上から何のグラフかを明記する]

[TODO: HH コンパートメントと Passive コンパートメントの区別について書く。HH コンパートメントは I_{ion} がスパイクの発射を駆動し、Passive コンパートメントは $C_m dV/dt = -g_{\text{leak}}(V - E_{\text{rest}}) + I_{\text{ext}}$ のみで受動的に振る舞う]

2.4 リカレントニューラルネットワーク

2.4.1 時系列データと過去の隠れ層

RNN は、並びに規則性のあるデータを学習することで、未知の時系列データが与えられたときにその未来の状態を予測するモデルである。時系列データを扱うためには、過去の状態をモデル内に保持し、現在に対する過去からの影響を把握しておく必要がある。一般的なニューラルネットワークは「入力層 $\mathbf{x}(t)$ - 隠れ層 $\mathbf{h}(t)$ - 出力層 $\mathbf{y}(t)$ 」という構造を持つが、RNN ではこれに加えて時刻 $t-1$ における隠れ層の値 $\mathbf{h}(t-1)$ を保持しておき、それも時刻 t の隠れ層に伝える [11]。時刻 t の状態を $t-1$ の状態として保持しフィードバックさせるため、 $\mathbf{h}(t-1)$ には再帰的に過去の状態がすべて反映されていることになる。これがリカレント（再帰型）ニューラルネットワークと呼ばれる理由である。

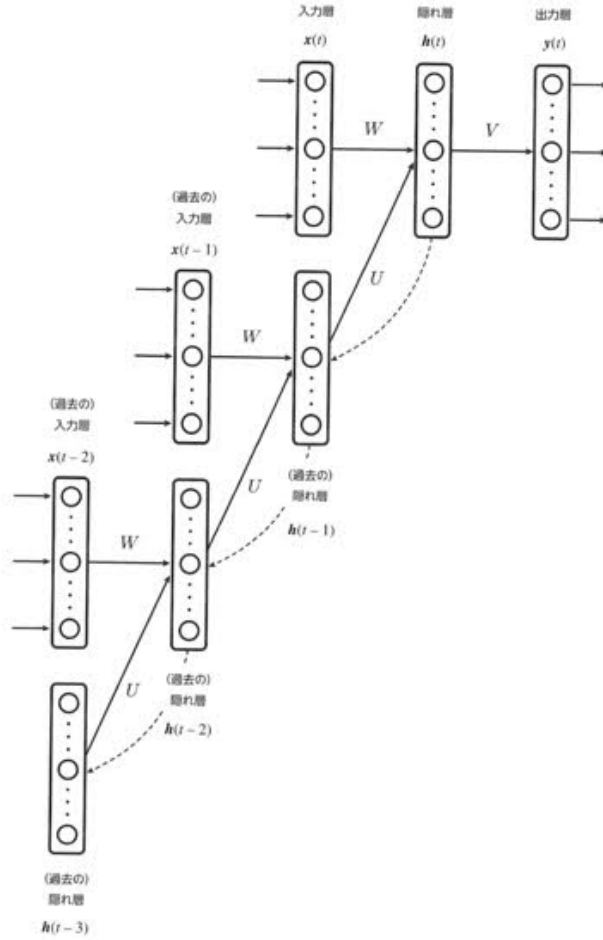


図 2.2 過去の隠れ層を加えたニューラルネットワークの構造。

2.4.2 モデル化

最も基本的な RNN (Elman 型 [12]) では、隠れ層と出力層は次式で表される。

$$\mathbf{h}(t) = f(W\mathbf{x}(t) + U\mathbf{h}(t-1) + \mathbf{b}) \quad (2.8)$$

$$\mathbf{y}(t) = g(V\mathbf{h}(t) + \mathbf{c}) \quad (2.9)$$

ここで $f(\cdot), g(\cdot)$ は活性化関数、 $\mathbf{b}, \mathbf{c}$ はバイアスペクトルである。 W は入力層から隠れ層への重み、 U は過去の隠れ層から隠れ層への重み (再帰重み)、 V は隠れ層から出力層への重みを表す。隠れ層の式に過去から順伝播してくる項 $U\mathbf{h}(t-1)$ が付くが、それ以外は一般的なニューラルネットワークと違いはない。そのため、各パラメータも誤差逆伝播法により最適化できる。

再帰計算を含む場合、活性化関数 f には注意が必要である。通常のニューラルネットワークでは ReLU が効果的であるが、ReLU は出力が入力に比例し最大値が $+\infty$ であるため、同じ重みが繰り返し掛けられる再帰計算ではオーバーフローを起こす可能性がある。そのため、出力値が一定範囲に収まる双曲線正接関数 $\tanh$ が用いられるのが一般的である [11]。本研究でも $f = \tanh$ とする。

2.4.3 Backpropagation Through Time

誤差関数を E とし、隠れ層・出力層の活性化前の値をそれぞれ

$$\mathbf{p}(t) := W\mathbf{x}(t) + U\mathbf{h}(t-1) + \mathbf{b} \quad (2.10)$$

$$\mathbf{q}(t) := V\mathbf{h}(t) + \mathbf{c} \quad (2.11)$$

と定義する。これに対する誤差項を

$$\mathbf{e}_h(t) := \frac{\partial E}{\partial \mathbf{p}(t)}, \quad \mathbf{e}_o(t) := \frac{\partial E}{\partial \mathbf{q}(t)} \quad (2.12)$$

とすると、各パラメータの勾配は次のように表せる。

$$\begin{aligned} \frac{\partial E}{\partial W} &= \mathbf{e}_h(t) \mathbf{x}(t)^\top & \frac{\partial E}{\partial \mathbf{b}} &= \mathbf{e}_h(t) \\ \frac{\partial E}{\partial U} &= \mathbf{e}_h(t) \mathbf{h}(t-1)^\top & \frac{\partial E}{\partial \mathbf{c}} &= \mathbf{e}_o(t) \\ \frac{\partial E}{\partial V} &= \mathbf{e}_o(t) \mathbf{h}(t)^\top \end{aligned} \quad (2.13)$$

ここで注意すべき点がある。一般的なニューラルネットワークでは、誤差関数を 2 乗誤差関数とする場合、誤差項は

$$\mathbf{e}_h(t) = f'(\mathbf{p}(t)) \odot V^\top \mathbf{e}_o(t) \quad (2.14)$$

$$\mathbf{e}_o(t) = g'(\mathbf{q}(t)) \odot (\mathbf{y}(t) - \mathbf{t}(t)) \quad (2.15)$$

で与えられる。これらの式自体は正しいが、RNN ではネットワークの順伝播で時刻 $t-1$ における隠れ層の出力 $\mathbf{h}(t-1)$ を考えたため、逆伝播の際も $t-1$ における誤差を考える必要がある。すなわち、順伝播において $\mathbf{h}(t)$ が $\mathbf{h}(t-1)$ の再帰関係式で表されたのと同様に、逆伝播では $\mathbf{e}_h(t-1)$ を $\mathbf{e}_h(t)$ の式で表す必要がある。このとき誤差は時間を遡って逆伝播していることになるので、これを Backpropagation Through Time (以下 BPTT) と呼ぶ [13]。

$\mathbf{e}_h(t-1) = \partial E / \partial \mathbf{p}(t-1)$ であることから、再帰関係式を求めると

$$\begin{aligned} \mathbf{e}_h(t-1) &= \frac{\partial E}{\partial \mathbf{p}(t)} \odot \frac{\partial \mathbf{p}(t)}{\partial \mathbf{p}(t-1)} \\ &= \mathbf{e}_h(t) \odot \left(\frac{\partial \mathbf{p}(t)}{\partial \mathbf{h}(t-1)} \frac{\partial \mathbf{h}(t-1)}{\partial \mathbf{p}(t-1)} \right) \\ &= \mathbf{e}_h(t) \odot (U f'(\mathbf{p}(t-1))) \end{aligned} \quad (2.16)$$

が得られる。よって、再帰的に

$$\mathbf{e}_h(t-z-1) = \mathbf{e}_h(t-z) \odot (U f'(\mathbf{p}(t-z-1))) \quad (2.17)$$

の関係で表すことができる。これによりすべての勾配が計算できるため、各パラメータの更新式は学習率を η として次のようになる。

$$\begin{aligned}
W(t+1) &= W(t) - \eta \sum_{z=0}^{\tau} \mathbf{e}_h(t-z) \mathbf{x}(t-z)^{\top} \\
U(t+1) &= U(t) - \eta \sum_{z=0}^{\tau} \mathbf{e}_h(t-z) \mathbf{h}(t-z-1)^{\top} \\
V(t+1) &= V(t) - \eta \mathbf{e}_o(t) \mathbf{h}(t)^{\top} \\
\mathbf{b}(t+1) &= \mathbf{b}(t) - \eta \sum_{z=0}^{\tau} \mathbf{e}_h(t-z) \\
\mathbf{c}(t+1) &= \mathbf{c}(t) - \eta \mathbf{e}_o(t)
\end{aligned} \tag{2.18}$$

この τ がどれくらいの過去まで遡って時間依存性を見るか（すなわちデータ長）を表すので、理想的には $\tau \rightarrow +\infty$ とすべきである。しかし、現実には勾配が消失（あるいは爆発）してしまうことを防ぐため、 $\tau = 10 \sim 100$ 程度に設定するのが一般的である [11]。

2.4.4 再帰計算がある場合の重みの初期値

ネットワークの順伝播を考えると、隠れ層の再帰部分には同じ重み U が繰り返し掛けられることになる。そのため、通常のニューラルネットワークの初期化手法（Xavier の初期化など）を用いると、計算中にオーバーフローを起こす可能性が高くなる。

そこで用いられるのが直交行列による初期化である [14]。直交行列とは、正方行列 U に対して $UU^{\top} = I$ 、すなわち $U^{-1} = U^{\top}$ となる行列を指す。重みが直交行列として初期化されていれば、再帰計算を行ってもオーバーフローを起こしにくくなるため、計算の安定化が見込める。

第 3 章

RNN による代理モデルの構築手法

3.1 概要

本研究では、外部電流 $I_{\text{ext}}(t)$ に対する膜電位 $V(t)$ の応答を、微分方程式を数値積分する代わりに RNN の推論によって再現する代理モデルを構築する。具体的な手順は次のとおりである。

1. HH モデルのシミュレーションを陽的 Euler 法により行い、ランダムな外部入力 I_{ext} に対する $V(t), m(t), h(t), n(t)$ の時系列データを取得する (3.2 節)。
2. 得られた時系列データを、RNN が学習できる形式 (一定長の系列とその直後の値の組) に変換する (3.3 節)。
3. NumPy のみを用いて実装した RNN に、このデータを学習させる (3.4 節)。
4. 学習済みの RNN に外部電流を与え、膜電位応答を逐次的に生成させる (3.5 節)。
5. 生成された膜電位応答を、元の HH モデルの応答と比較して評価する (3.6 節)。

[TODO: RNN に何を入力し何を出力させるかが最大の設計上の論点であり、まだ確定していない。現時点で考えている候補を以下に挙げる。実験して決める。

- (a) 入力 I_{ext} の系列 $\rightarrow$ 出力 V (最も素朴。隠れ変数を完全に隠蔽できる)
- (b) 入力 (I_{ext}, V) の系列 $\rightarrow$ 出力 次時刻の V (自己回帰。学習は容易だが誤差が累積する)
- (c) 入力 $(I_{\text{ext}}, V, m, h, n) \rightarrow$ 出力 次時刻の (V, m, h, n) (微分方程式の右辺を学習させる形)
- (d) 一部を HH モデル、一部を RNN で置き換えるハイブリッド]

3.2 シミュレーション条件

3.2.1 Euler 法

HH モデルの実体は n 元 1 階連立常微分方程式

$$\frac{dx}{dt} = f(x, t) \quad (3.1)$$

である。今回扱うモデルにおいては f が非線形の関数であるため、解析解を求めることは非常に困難である。そこで、シミュレーションの手法として数値計算手法の一つである陽的 Euler 法を用

いた。Euler 法では、区間 $[a, b]$ を刻み幅 h で等間隔に n 等分した点 $t_0, t_1, \dots, t_n$ に対し、各点での近似解 $x_0, x_1, \dots, x_n$ を Algorithm 1 のように求める [8]。

Algorithm 1 Euler 法

Require: 初期値 c_0 、刻み幅 h

```

1:  $x_0 \leftarrow c_0$ 
2: for  $i = 0, 1, \dots, n - 1$  do
3:    $x_{i+1} \leftarrow x_i + h f(x_i, t_i)$ 
4: end for
5: return  $x_0, x_1, \dots, x_n$ 

```

3.2.2 初期値の与え方

膜電位の初期値は静止膜電位 $V_{\text{rest}} = -65.0$ [mV] を与える。[**TODO: 本研究の HH 実装では V の基準をどう取っているか (-65 mV か 0 mV か) を確認して統一する。式 (2.6) のレート関数は V の基準に依存する**]

ゲート変数の初期値については、式 (2.5) を変形すると

$$\frac{1}{\alpha_x(V) + \beta_x(V)} \frac{dx}{dt} = - \left(x - \frac{\alpha_x(V)}{\alpha_x(V) + \beta_x(V)} \right) \quad (3.2)$$

となる。この式は、ゲート変数 x が $\alpha_x(V)/(\alpha_x(V) + \beta_x(V))$ において $dx/dt = 0$ 、すなわち定常状態になることを意味する。時刻 $t = 0$ の初期値において傾きが 0 であってほしいので、ゲート変数の初期値は

$$\begin{pmatrix} m_0 \\ h_0 \\ n_0 \end{pmatrix} = \begin{pmatrix} \alpha_m(V_0)/(\alpha_m(V_0) + \beta_m(V_0)) \\ \alpha_h(V_0)/(\alpha_h(V_0) + \beta_h(V_0)) \\ \alpha_n(V_0)/(\alpha_n(V_0) + \beta_n(V_0)) \end{pmatrix} \quad (3.3)$$

とする。

3.2.3 外部電流の生成方法

教師データには、なるべく様々な入力とそれに対する応答が含まれることが望ましい。様々な入力が教師データの中に存在すれば、特定の入力に依存しないロバストなモデルの構築が期待できる。そこで、教師データに用いる外部電流 I_{ext} は、ランダムな振幅を持つパルス電流の組み合わせとして与える (Algorithm 2)。

[**TODO: パラメータを確定させる。先行研究 [10] では $\Delta = 10$ ms、 $I_{\text{max}} = 20$ mA、 $dt = 0.05$ ms、 $T_{\text{sim}} = 900$ ms、最新の進捗報告では $-5 \sim 20$ mA、 20 ms 幅、 $dt = 0.01$ ms が使われている。本研究でどの条件を使うかを決め、その理由も書く**]

Algorithm 2 I_{ext} の決定

Require: シミュレーション時間 T_{sim} 、時間間隔 $\Delta (\ll T_{\text{sim}})$ 、電流の範囲 $[I_{\text{min}}, I_{\text{max}}]$

- 1: 区間 $[0, T_{\text{sim}}]$ を Δ で分割し、 $t_0 = [0, \Delta], t_1 = [\Delta, 2\Delta], \dots$ を得る
 - 2: **for** $i = 0, 1, \dots, n$ **do**
 - 3: $[I_{\text{min}}, I_{\text{max}}]$ の中から一様乱数で I_{rnd} を選ぶ
 - 4: 区間 t_i の間は I_{rnd} [mA] の定常電流が流れているものとする
 - 5: **end for**
 - 6: **return** $I_{\text{ext}} = [I_{\text{ext}}^{(0)}, I_{\text{ext}}^{(1)}, \dots, I_{\text{ext}}^{(n)}]$
-

3.3 学習データの作成

RNN の学習には、一定長の入力系列とその直後の正解値の組が必要である。系列長を τ として、時系列データ $f(0), f(1), \dots$ を次のように分割する。

$$[f(t), f(t+1), \dots, f(t+\tau-1)] \longrightarrow f(t+\tau) \quad (3.4)$$

すなわち、 τ 点の系列を入力とし、その次の 1 点を予測する教師あり学習とする。窓を 1 時刻ずつずらしながらこの組を作るため、元の時系列の長さを L とするとサンプル数は $L - \tau$ となる。入力データ X と正解データ t の形状はそれぞれ

$$X \in \mathbb{R}^{(L-\tau) \times \tau \times D}, \quad t \in \mathbb{R}^{(L-\tau) \times K} \quad (3.5)$$

となる (D は入力次元、 K は出力次元)。一定長に分割するのは BPTT における計算の都合という理由もあるが、データ数を増やして学習できるという理由もある [11]。

得られたデータは訓練データと検証データに分割する。ただし時系列データを扱う場合、未来の時刻のデータが訓練データに含まれないようにしなければならない。すなわち、全データをランダムにシャッフルするのではなく、前から順に一定数を訓練データ、残りを検証データとする。本研究では訓練データ 80%、検証データ 20% とした。

[TODO: 膜電位は $-65 \sim +40$ mV 程度、ゲート変数は $0 \sim 1$ とスケールが大きく異なるため、正規化（標準化）が必要になると思われる。どの正規化を行ったかをここに書く]

3.4 RNN の実装

本研究では、RNN の内部計算を理解することを目的として、PyTorch などのフレームワークを使わず NumPy の基本的な配列演算のみで RNN を実装した。以下、実装した処理を順に述べる。

3.4.1 順伝播

時刻 $t = 0$ には 1 つ前の時刻が存在しないため、隠れ状態の初期値は $h(-1) = \mathbf{0}$ とする。その後、式 (2.8) に従って時刻ごとに隠れ状態を更新する。実装では、バッチを行方向に並べるため、資

料の式とは行列の左右が入れ替わった形

$$P = X_t W + H_{t-1} U + \mathbf{b}, \quad H_t = \tanh(P) \quad (3.6)$$

で計算している。出力は最後の時刻の隠れ状態のみを用い、回帰問題であるため活性化関数は恒等関数とする。

$$Y = H_{\tau-1} V + \mathbf{c} \quad (3.7)$$

3.4.2 損失関数

膜電位の回帰問題であるため、損失関数には平均二乗誤差を用いる。

$$E = \frac{1}{NK} \sum_{i=1}^N \sum_{k=1}^K (y_{ik} - t_{ik})^2 \quad (3.8)$$

3.4.3 勾配計算と最適化

勾配は式 (2.14)～(2.17) に従って BPTT により計算する。実装が正しいことを確認するため、中心差分による数値微分

$$\frac{\partial E}{\partial w} \simeq \frac{E(w + \epsilon) - E(w - \epsilon)}{2\epsilon} \quad (3.9)$$

と比較する検算を行った (4.1 節)。

パラメータの更新には、まず式 (2.18) の最も基本的な勾配降下法を用いた。その後、必要に応じて Adam[15] を用いる。**[TODO: 最終的にどちらを採用したか、その理由 (sin 波の検証では Adam でないと自己回帰生成が破綻した) を結果の節と対応させて書く]**

3.5 代理モデルとしての推論方法

学習済みの RNN を代理モデルとして使う際は、モデル自身の予測値を次の入力に戻しながら逐次的に膜電位応答を生成する。すなわち、最初の τ 点のみを与え、以降は

1. 現在の τ 点の系列から次の 1 点を予測する
2. 予測値を系列の末尾に加え、先頭の 1 点を捨てる

を繰り返す。この方法では、本物のデータを一切参照せずに応答を生成することになるため、モデルがダイナミクスそのものを学習できているかを厳しく評価できる。一方で予測の誤差が累積するため、長時間のシミュレーションでは波形が崩れる可能性がある。

3.6 評価指標

構築した代理モデルを、元の HH モデルと次の観点で比較する。評価指標は先行研究の進捗報告 [10] で整備されたものを踏襲する。

3.6.1 波形の誤差

同じ外部電流を与えたときの膜電位波形について、平均二乗誤差の平方根 (RMSE) と平均絶対誤差 (MAE) を計算する。

$$\text{RMSE} = \sqrt{\frac{1}{L} \sum_{t=1}^L (V^{\text{surr}}(t) - V^{\text{orig}}(t))^2}, \quad \text{MAE} = \frac{1}{L} \sum_{t=1}^L |V^{\text{surr}}(t) - V^{\text{orig}}(t)| \quad (3.10)$$

3.6.2 スパイク単体の形状

スパイク 1 本の形状的な特徴について、次の指標で定量比較する。

- latency (電流を与えてから発火するまでの時間)
- AP_amplitude (活動電位の振幅)
- AP_begin_voltage (発火が始まる膜電位)
- AP_duration_half_width (振幅の半分の高さにおけるスパイク幅)

3.6.3 入力条件を変えた応答

学習データに含まれない入力に対しても正しく応答できるか (外挿性能) を評価する。

- 定常電流の振幅を掃引したときのスパイク数 (発火頻度) の推移
- ramp 電流を与えたときの発火閾値
- sin 波電流に対する応答

3.6.4 計算コスト

シミュレーションに必要な計算コストを次の項目で比較する。

- 空間計算量: シミュレーションの際に保持する必要がある浮動小数点数の個数。HH モデルでは V, m, h, n の 4 つ、RNN では隠れ状態 $\mathbf{h}$ の次元数に対応する。
- 時間計算量: 1 タイムステップの計算に必要な浮動小数点演算数・指数演算数。
- 実計算時間: 一定時間のシミュレーションを行った際の実際の計算時間。

[TODO: RNN の場合、隠れ状態の次元 (例: 50) が HH モデルの状態変数の数 (4) より多

くなるため、単純な比較では空間計算量が増えてしまう。「1 コンパートメントあたり」ではなく「ニューロン全体・ネットワーク全体で」どう比較するのが妥当かを考える必要がある。また、RNNは重み行列を保持する必要があるが、これは全コンパートメントで共有できるため、コンパートメント数に比例しないという利点がある。この議論を整理して書く]

第 4 章

結果と考察

4.1 sin 波による RNN 実装の検証

HH モデルのデータを扱う前に、実装した RNN が正しく動作することを sin 波の予測問題で確認した。周期 $T = 100$ の sin 波に振幅 0.05 の一様乱数によるノイズを加えた時系列

$$f(t) = \sin\left(\frac{2\pi}{T}t\right) + 0.05u, \quad u \sim U(-1, 1) \quad (4.1)$$

を $t = 0, \dots, 2T$ について生成し、 $\tau = 25$ として式 (3.4) の形式に変換した。データの形状は $X \in \mathbb{R}^{176 \times 25 \times 1}$ 、 $t \in \mathbb{R}^{176 \times 1}$ となる。隠れ層の次元は 50 とした。

BPTT の実装が正しいことを確認するため、式 (3.9) の数値微分と比較した。結果を表 4.1 に示す。いずれのパラメータについても相対誤差は 10^{-7} 以下であり、BPTT の実装は正しいことが確認できた。

表 4.1 BPTT による勾配と数値微分による勾配の比較

パラメータ	最大相対誤差
W	4.5×10^{-11}
U	3.0×10^{-7}
$\mathbf{b}$	2.6×10^{-10}
V	1.1×10^{-10}
$\mathbf{c}$	8.5×10^{-13}

学習は 1000 エポック、バッチサイズ 100 で行った。損失は訓練・検証ともに 0.0013 程度で下げ止まった。これは学習データに加えたノイズの分散 $0.05^2/3 \simeq 8.3 \times 10^{-4}$ とほぼ同じ水準であり、ノイズ成分は原理的に学習できないため、これ以上損失は下がらない。

3.5 節の方法で最初の 25 点のみから波形を生成させた結果を図 4.1 に示す。ノイズなしの sin 波とほぼ一致する波形を生成できており、実装した RNN が時系列の周期性を学習できていることが分かる。

なお、パラメータ更新に最も基本的な勾配降下法を用いた場合、次の 1 点の予測（損失の低下）自

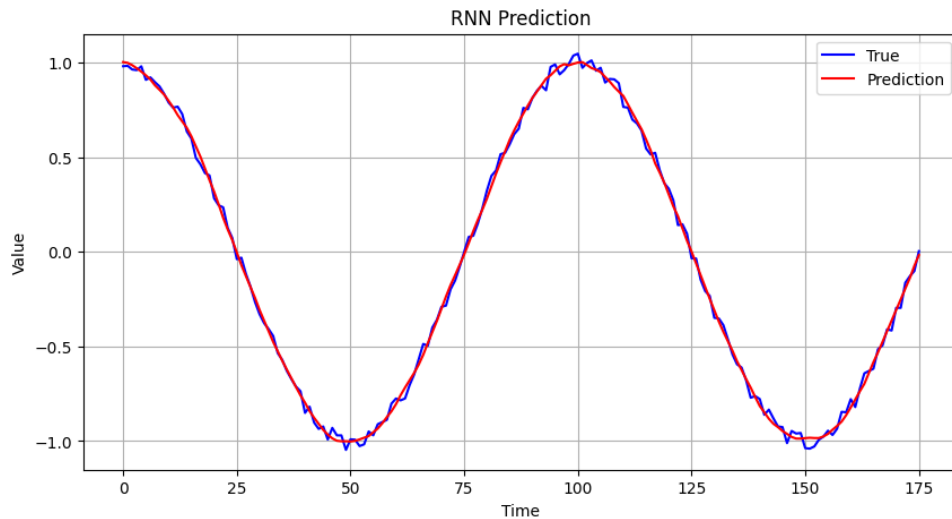


図 4.1 実装した RNN による sin 波の生成。[TODO: NumPy 実装での最新の図 (Python/RNN/prediction.png) に差し替える]

体は達成できたが、予測値を入力に戻して波形を生成させると形が崩れてしまった。Adam を用いることでこの問題は解消した。[TODO: この比較結果 (表) を載せる。README.md に測定値がある]

4.2 Hodgkin-Huxley モデルのシミュレーション

[TODO: Algorithm 2 で生成した外部電流と、それに対する HH モデルの応答 (V, m, h, n) の図を載せる。先行研究の図 4.1 に相当するもの]

4.3 学習結果

[TODO: 学習曲線 (訓練損失・検証損失) と、最終的な損失の値を載せる]

4.4 代理モデルの評価

[TODO: 3.6 節の各指標について結果を載せる。(1) 波形の比較図と RMSE・MAE (2) スパイク特徴量の比較表 (3) 定常電流振幅の掃引に対するスパイク数、ramp 電流に対する閾値、sin 波応答 (4) 計算コストの比較表]

4.5 考察

[TODO: 以下の観点で書く。・RNN が HH モデルの膜電位応答をどこまで再現できたか・SINDy による先行研究 [10] との比較。SINDy は微分方程式の形で解釈可能なモデルが得られる一方、時間計算量が増加した。RNN は解釈性に劣るが行列演算のみで構成されるため GPU によ

る並列化に向く、という対比ができるはず・自己回帰による誤差累積の影響・学習データに含まれない入力に対する外挿性能の限界・Multi-Compartment モデルへ拡張した場合に期待される計算コスト削減量の見積もり]

第 5 章

まとめ

[TODO: 以下の流れで書く。・本研究は RNN を用いてニューロンモデルの代理モデルを構築した・RNN は PyTorch を使わず NumPy のみで実装し、BPTT が正しいことを数値微分で確認した・(結果のまとめ)・今後は Multi-Compartment モデルに対する代理モデルの構築を行う]

謝辞

本研究を進めるにあたり、多大なるご指導とご助言を賜りました小林泰良助教に深く感謝申し上げます。[TODO: 使用したライブラリ (NumPy, Matplotlib など) への謝辞、先行研究の実装を参考にさせていただいた点への謝辞を書く]

参考文献

- [1] 山崎 匡, 五十嵐 潤. はじめての神経回路シミュレーション: 1 ニューロンからヒト全脳モデルまで. 森北出版株式会社, 2021, pp. 56–61.
- [2] Eric R. Kandel et al. カンデル神経科学. 第 2 版. メディカル・サイエンス・インターナショナル, 2022, p. 57.
- [3] Albert Gidon et al. Dendritic Action Potentials and Computation in Human Layer 2/3 Cortical Neurons. *Science*, Jan. 2020. doi: 10.1126/science.aax6239.
- [4] Tiago Branco, Beverley A. Clark, Michael Häusser. Dendritic Discrimination of Temporal Input Sequences in Cortical Neurons. *Science*, 329.5999 (Sept. 2010), p. 1671. doi: 10.1126/science.1189664.
- [5] Kobayashi Taira, Tamura Akira, Rin Kuriyama. Development of a lightweight and customizable biophysical neuron simulator. 2024. <https://researchmap.jp/tairakobayashi/presentations/48836360>
- [6] A. L. Hodgkin, A. F. Huxley. A Quantitative Description of Membrane Current and Its Application to Conduction and Excitation in Nerve. *The Journal of Physiology*, 117.4 (Aug. 1952), pp. 500–544. doi: 10.1113/jphysiol.1952.sp004764.
- [7] 宮川 博義, 井上 雅司. ニューロンの生物物理. 第 2 版. 丸善出版, 2013.
- [8] 峯村 吉泰. C と Java で学ぶ数値シミュレーション入門. 森北出版株式会社, 2008, pp. 155–157.
- [9] Steven L. Brunton, Joshua L. Proctor, J. Nathan Kutz. Discovering Governing Equations from Data: Sparse Identification of Nonlinear Dynamical Systems. Sept. 2015. doi: 10.1073/pnas.1517384113.
- [10] 棟近 春樹. Hodgkin-Huxley モデルの計算コスト削減を目指したサロゲートモデルの開発. 卒業論文, 山口大学理学部物理・情報科学科, 令和 6 年度.
- [11] 巢籠 悠輔. 詳解ディープラーニング 第 2 版: TensorFlow/Keras・PyTorch による時系列データ処理. マイナビ出版, 2019.
- [12] Jeffrey L. Elman. Finding Structure in Time. *Cognitive Science*, 14.2 (1990), pp. 179–211. doi: 10.1207/s15516709cog1402_1.
- [13] Paul J. Werbos. Backpropagation Through Time: What It Does and How to Do It. *Proceedings of the IEEE*, 78.10 (Oct. 1990), pp. 1550–1560. doi: 10.1109/5.58337.
- [14] Andrew M. Saxe, James L. McClelland, Surya Ganguli. Exact Solutions to the Nonlinear Dynamics of Learning in Deep Linear Neural Networks. 2014. arXiv: 1312.6120.

- [15] Diederik P. Kingma, Jimmy Ba. Adam: A Method for Stochastic Optimization. 2015. arXiv: 1412.6980.
- [16] Ricky T. Q. Chen et al. Neural Ordinary Differential Equations. Dec. 2019. doi: 10.48550/arXiv.1806.07366.

付録 A

付録

A.1 実装したコード

[TODO: NumPy による RNN の実装 (Python/RNN/simple_rnn.py) の主要部分を載せるか、リポジトリの URL を記載する]